OpenDeepResearcher

Agentic LLM Research Framework

By : Yashvardhan Singh

Project Objective

The primary objective of the OpenDeepResearcher project is to design and implement an intelligent system capable of autonomously performing structured research tasks in a manner similar to a human researcher. The system aims to:

- Understand complex user queries
- Decompose them into smaller, researchable sub-questions
- Retrieve relevant and high-quality information from the internet
- Process and summarize the retrieved data
- Present the results in a structured and user-friendly format

Additionally, the project focuses on building a **scalable and modular architecture** that allows easy integration of future enhancements such as citation ranking, PDF export, and advanced reasoning capabilities.

Development Environment Setup

A robust Python-based development environment was established to support the implementation of the system. The setup ensures reproducibility, dependency management, and smooth integration of all required libraries and tools.

Steps Performed:

- Created a virtual environment:
- `python -m venv .venv`
- Activated the environment:
- `.venv\Scripts\activate`
- Installed dependencies:
- `pip install -r requirements.txt`

Tools and Technologies Used:

- **Python** for core development
- **Streamlit** for user interface
- **LM Studio** for running local LLMs
- **Tavily API** for web search
- **JSON** for structured data storage

System Architecture Overview

The OpenDeepResearcher system follows a **modular multi-agent architecture**, where each agent performs a specialized task within the research workflow. This design ensures separation of concerns, scalability, and ease of debugging

Architecture Flow:

```
User Query (Streamlit UI)
        ↓
Planner Agent (Sub-question Generation)
        ↓
Researcher Agent (Web Data Retrieval)
        ↓
Writer Agent (Summarization &
Structuring)
        ↓
JSON Storage (research_data.json)
        ↓
Final Output Display (UI)
```

Key Characteristics:

- Modular design
- Agent-based workflow
- JSON-based data pipeline
- Context-aware processing
- Real-time interaction

Milestone 1 – Foundation Setup

The first milestone focused on building a strong foundation by understanding the system architecture and preparing the development environment.

Key Activities:

- Studied agent-based AI architectures
- Understood roles of Planner, Researcher, and Writer agents
- Explored LLM capabilities and prompt engineering
- Configured LM Studio for local model execution
- Tested instruction-based models for response generation

Outcome:

A clear architectural blueprint and a fully functional development environment ready for implementation.

Milestone 2 – Core Agent Development

This milestone involved the implementation of the core agents and the establishment of the research pipeline.

Planner Agent

- Accepts user query
- Generates 5 structured sub-questions
- Stores them in JSON format

Researcher Agent

- Uses Tavily API for web search
- Retrieves multiple relevant sources
- Extracts content, URLs, and titles

Writer Agent

- Processes raw research data
- Generates structured summaries
- Ensures clarity, readability, and completeness

Pipeline Workflow

1. User inputs query
2. Planner generates sub-questions
3. JSON structure is created
4. Researcher fills raw data
5. Writer summarizes content
6. Final output is generated

Outcome:A fully working backend system capable of performing automated research tasks.

Milestone 3 – User Interface & Interaction Layer

This milestone focused on transforming the backend system into an interactive application using Streamlit.

Features Implemented:

- Chat-based interface
- Dynamic message rendering
- Sidebar with chat history
- Multi-session chat management
- Automatic chat naming

Benefits:

- Improved user experience
- Interactive research workflow
- Easy navigation between session

Milestone 4 – Advanced Interaction, Memory & Robustness

This milestone represents the transition of the system into a **real-world intelligent assistant** by introducing advanced interaction capabilities and system robustness.

1. Context-Aware Query Handling

The system can now understand whether a new query is related to previous conversation context.

Implementation:

- LLM-based relation classification
- Uses recent chat history

Impact:

- Enables follow-up questions
- Maintains conversation continuity

2. Multi-Session Chat Memory

Users can manage multiple independent research sessions.

Features:

- Separate memory per chat

- Easy switching between sessions
- Persistent conversation history

3. Duplicate Output Prevention

A major issue of repeated responses was resolved.

Solution:

- Introduced processed query tracking
- Ensured each query is processed only once

Result:

- Clean and stable output
- No redundant responses

4. Retry and Error Recovery System

The system can gracefully handle failures and allow retry without retyping queries.

Errors Handled:

- LLM connection issues
- Missing API keys
- Internet failures

Features:

- Reload button
- Automatic query recovery

5. Real-Time Progress Tracking

Users are informed about the processing stages.

Stages:

- Planning
- Researching
- Writing

Benefit:

- Transparency
- Better user engagement

6. JSON-Based Data Pipeline

The entire workflow is structured using a JSON file.

Advantages:

- Clean data flow
- Easy debugging
- Scalable design

7. Enhanced UI/UX Design

Improvements were made to create a professional interface.

Additions:

- Fixed input bar
- Styled buttons
- Sticky header
- Organized layout

8. Robust Exception Handling

Custom error handling ensures system stability.

Benefits:

- Prevents crashes
- Provides clear error messages
- Improves reliability

Conclusion

The OpenDeepResearcher project has successfully evolved into a **fully functional, intelligent, and interactive AI research assistant**. Starting from basic architectural understanding to implementing advanced interaction features, the system now demonstrates the ability to perform complex research tasks autonomously while maintaining a high level of usability and robustness.

The integration of multi-agent architecture, context-aware processing, real-time interaction, and error resilience makes the system highly scalable and suitable for real-world applications. Future enhancements can further extend its capabilities, making it a powerful tool in the domain of AI-assisted research.